

AI-Powered Phishing Detector

Main Concepts & State of the Art

Team	Malek Ammar , Atef Hammemi , Mohamed Ali Ferchichi , Ali Zrida , Hafidh Aoun
Course	Information Assurance and Security
Teacher	Manel Abdelkader
Date	April 19, 2026
Type	University Project

Contents

1	Project Definition	2
1.1	Core Purpose	2
1.2	Target Audience	2
2	Main Concepts & Components	2
2.1	Email as a Data Structure	2
2.2	Text Preprocessing	3
2.3	Feature Extraction	3
2.4	Threat Vectors & Attack Patterns	4
2.5	Architectural Components	4
2.5.1	Ingestion Layer	4
2.5.2	Feature Engineering Pipeline	4
2.5.3	Classification Engine	5
2.5.4	Output & Alerting	5
3	Functional Flow	5
4	State of the Art	6
4.1	Classical ML Models (Scikit-learn)	6
4.2	Pre-trained NLP Models	7
4.3	Commercial / Cloud Solutions	7
4.4	Recommended Stack for This Project	7
5	Dataset Sources	8
6	Target Feature Schema	8

1 Project Definition

The AI-Powered Phishing Detector is an automated classification system that analyses incoming emails and determines whether each message is a phishing attempt (Phish) or a legitimate communication (Ham). The system leverages machine learning and natural language processing techniques to detect deceptive patterns that attackers — increasingly assisted by large language models — embed in fraudulent emails.

1.1 Core Purpose

The detector automates three core tasks:

- Classification — assigns each email a binary label (Phish / Ham) with a confidence score.
- Feature Extraction — identifies and quantifies signals such as urgency cues, suspicious hyperlinks, header anomalies, and sender mismatches.
- Explainability — surfaces the specific features that drove each classification decision, enabling analysts to audit and refine the model.

1.2 Target Audience

- Security Operations Centre (SOC) Analysts — need triage automation to handle high email volumes.
- IT Administrators — deploy the detector as a mail-gateway filter or an employee-facing reporting tool.
- End Users — receive real-time warnings before opening or clicking suspicious messages.
- Researchers — study LLM-generated phishing language, evasion strategies, and evolving attack patterns.

2 Main Concepts & Components

2.1 Email as a Data Structure

An email is composed of two distinct parts that the detector analyses independently:

- Headers — machine-readable metadata fields: From, Reply-To, Return-Path, Received, DKIM-Signature, SPF result, DMARC policy. Header inconsistencies are among the strongest phishing signals.
- Body — the human-readable content, which may be plain text, HTML, or a multipart mix. HTML bodies can hide the true destination of hyperlinks and embed tracking pixels.

Key parsing points:

- MIME decoding — base64 or quoted-printable encoded bodies must be decoded before analysis.
- HTML stripping — raw HTML tags are removed to expose visible text; href attributes are separately extracted for URL analysis.
- Attachment metadata — MIME type, filename extension, and hash fingerprint are recorded even if attachment content is not analysed.

2.2 Text Preprocessing

Raw email text must be normalised before feature extraction or model input:

- Lowercasing and whitespace normalisation — reduces vocabulary size; prevents duplicate tokens.
- Tokenisation — splits text into individual words or sub-word units (word-piece for transformer models).
- Stop-word removal — eliminates common words (the, and, is) that carry no discriminative signal for classical models.
- Stemming / Lemmatisation — reduces inflected forms to a common base (e.g., verifying, verified → verify).
- TF-IDF vectorisation — converts token lists to numerical vectors weighted by term frequency and inverse document frequency, the standard input format for Scikit-learn classifiers.

2.3 Feature Extraction

Feature extraction translates raw email content into a structured numerical vector. The table below documents each feature category:

Feature Category	Specific Features	Phishing Signal
Urgency / Emotion	Count of urgency keywords (urgent, immediately, expire, verify, suspended)	High count strongly correlated with phishing intent.
Suspicious Links	URL count; mismatched anchor text vs. href; use of URL shorteners; IP-address URLs; HTTP (non-HTTPS) links	Disguised or redirecting URLs are the primary phishing delivery vector.
Sender Mismatch	From domain vs. Reply-To domain; DKIM pass/fail; SPF pass/fail; display name spoofing distance (Levenshtein)	Spoofed or mismatched sender identity is a defining phishing characteristic.
Structural / HTML	Presence of hidden text; CSS font-size: 0; image-to-text ratio; form action URL	Hidden content evades keyword-based filters; forms harvest credentials.

Linguistic / NLP	Readability score; spelling error count; sentence length variance; use of personalisation tokens ({{FirstName}})	LLM-generated phishing tends to be grammatically perfect but impersonal.
Attachment Risk	Attachment present (bool); extension type; double extension (e.g., .pdf.exe); file size	Executable or macro-enabled attachments are common malware delivery methods.

2.4 Threat Vectors & Attack Patterns

Modern phishing attacks employ several evasion strategies that the detector must account for:

Attack Pattern	Description	Detection Approach
Spear Phishing	Personalised email targeting a specific individual using OSINT data.	Sender mismatch features + context-aware NLP model.
LLM-Generated Phish	Grammatically perfect email generated by GPT-class models; bypasses spell-check heuristics.	Pre-trained classifier fine-tuned on AI-generated phishing corpus.
Homograph Attack	Unicode lookalike characters in domain names (e.g., paypal1.com).	Unicode normalisation + Levenshtein distance to known-good domains.
HTML Obfuscation	Real URL hidden behind benign anchor text; zero-font invisible text.	HTML parser extracts raw href values independent of visible text.
Brand Impersonation	Mimics trusted brands (bank, government, tech company) in logo and tone.	Brand keyword list + domain mismatch check against known brand domains.

2.5 Architectural Components

The detector is divided into four logical layers, each with a distinct responsibility.

2.5.1 Ingestion Layer

The Ingestion Layer is the entry point. It:

- Accepts raw .eml files, MIME-formatted strings, or connects to a mailbox via IMAP/POP3.
- Decodes MIME multipart structures and extracts headers, plain-text body, HTML body, and attachments separately.
- Normalises character encoding (UTF-8) and strips binary content not relevant to text analysis.

2.5.2 Feature Engineering Pipeline

The Feature Engineering Pipeline transforms raw email parts into a numerical feature vector. It:

- Applies text preprocessing (tokenisation, stop-word removal, TF-IDF).
- Runs URL extraction and validation against threat intelligence feeds.
- Performs header authentication checks (DKIM, SPF, DMARC).
- Outputs a fixed-length feature vector passed to the Classification Engine.

2.5.3 Classification Engine

The Classification Engine applies the trained model to each feature vector. It:

- Loads a serialised Scikit-learn pipeline (TF-IDF + Naive Bayes / Random Forest) or a fine-tuned transformer checkpoint.
- Returns a binary label (Phish / Ham) and a probability score in [0, 1].
- Logs each decision with the feature weights that most influenced the output (SHAP values for tree models; attention scores for transformers).

2.5.4 Output & Alerting

- JSON report — structured record of label, confidence, and top contributing features.
- Email quarantine — flag or move high-confidence phishing messages to a quarantine folder.
- Dashboard — real-time statistics on phishing volume, top attack patterns, and model accuracy metrics.

3 Functional Flow

The table below describes the complete operational sequence of the detector, from email ingestion through to final reporting and alerting.

#	Stage	Description
1	Ingestion	Raw email received as .eml file, IMAP fetch, or API call. MIME structure decoded; headers, body, and attachments separated.
2	Header Parsing	From, Reply-To, Return-Path, Received chain, and authentication results (DKIM, SPF, DMARC) extracted and stored as structured fields.
3	HTML Rendering	HTML body parsed by BeautifulSoup. Visible text extracted; all href values collected independently of anchor text. Hidden text nodes flagged.
4	Text Preprocessing	Lowercasing, stop-word removal, stemming applied to body text. Output: clean token list and TF-IDF vector.
5	Feature Extraction	All feature categories computed: urgency score, URL risk score, sender mismatch flags, HTML anomaly flags, attachment risk score. Assembled into feature vector.
6	URL Validation	Each extracted URL checked against domain blacklists, WHOIS registration age, and redirect chain analysis.
7	Classification	Feature vector fed to trained model. Binary label (Phish / Ham) and confidence score [0.00 – 1.00] returned.
8	Explainability	SHAP values computed for tree-based models; top 5 contributing features extracted and attached to the result record.
9	Thresholding	Score ≥ 0.85 : quarantine. 0.60–0.84: flag for analyst review. < 0.60 : deliver as Ham. Thresholds configurable.
10	Persistence	Result record inserted into SQLite database: label, score, feature values, timestamp, analyst override (if any).
11	Alerting	High-confidence phishing triggers email or Slack notification to SOC analyst. Webhook available for SIEM integration.
12	Logging	All decisions, model version, and errors written to a timestamped structured log file for auditing and model retraining.

4 State of the Art

4.1 Classical ML Models (Scikit-learn)

Model	Complexity	Typical Accuracy	Best Used For
Naive Bayes (MultinomialNB)	Low	92 – 95 %	Baseline TF-IDF classifier; fast training; highly interpretable.
Logistic Regression	Low	94 – 96 %	Linear decision boundary on TF-IDF features; good calibrated probabilities.
Random Forest	Medium	97 – 98 %	Mixed feature types (text + structural); SHAP explainability.
Gradient Boosting (XGBoost)	Medium-High	97 – 99 %	Highest accuracy on tabular feature vectors; handles class imbalance well.

Naive Bayes is the standard entry-point model for text classification. Its conditional independence assumption holds surprisingly well for bag-of-words representations of email text, and it trains in seconds even on large corpora.

Random Forest and XGBoost are the recommended production models when mixed feature vectors (TF-IDF + structural + header features) are used. Both natively support SHAP-based explainability.

4.2 Pre-trained NLP Models

Model	Architecture	JS Support	Notes
BERT (bert-base-uncased)	Transformer encoder	Via ONNX	Fine-tune on phishing corpus; understands context and semantic nuance.
DistilBERT	Distilled Transformer	Via ONNX	40 % smaller than BERT; 97 % of accuracy; preferred for resource-constrained environments.
RoBERTa	Optimised BERT	Via ONNX	Improved pre-training; state-of-the-art on many NLP benchmarks; robust to LLM-generated text.
SetFit (Few-Shot)	Sentence Transformer	No	Achieves high accuracy with as few as 8 labelled examples per class; ideal for novel attack variants.

4.3 Commercial / Cloud Solutions

Microsoft Defender for Office 365 is an enterprise-grade email security platform with AI-based anti-phishing policies, impersonation detection, and Safe Links URL detonation. It represents the industry gold standard but is beyond the scope of an academic project.

Google Safe Browsing is a public API that checks URLs against Google's continuously updated phishing and malware database. It is freely accessible and can be integrated directly into the URL validation step.

VirusTotal API aggregates results from 70+ antivirus engines and URL scanners. It provides a reputation score for any URL or file hash and is useful for enriching the URL validation step at no cost (within free-tier limits).

4.4 Recommended Stack for This Project

Scikit-learn with a TF-IDF + Random Forest pipeline for the classical baseline, DistilBERT fine-tuned on a phishing corpus for the NLP model, BeautifulSoup for HTML parsing, and SQLite for result persistence. This stack balances accuracy, explainability, and a manageable learning curve appropriate for a university project.

5 Dataset Sources

The following publicly available datasets were selected as training and evaluation sources based on their size, labelling quality, and coverage of diverse phishing styles.

Dataset	Size	Phish %	Notes
SpamAssassin Public Corpus	~6 000 emails	31 %	Classic benchmark; mix of spam, phishing, and ham. Plain-text and HTML emails included.
Enron Email Dataset	~500 000 emails	~10 %	Real corporate email; excellent ham baseline. Requires phishing injection for balanced training.
CEAS 2008 Spam Corpus	~38 000 emails	~50 %	Balanced spam/ham corpus from the 2008 Conference on Email and Anti-Spam challenge.
Phishing Pot (OpenPhish)	Continuous feed	100 %	Live phishing URL and email feed. Useful for up-to-date LLM-generated phishing samples.
Ling-Spam Dataset	2 893 emails	16 %	Annotated academic mailing-list corpus; useful for evaluating false-positive rates on legitimate technical emails.

6 Target Feature Schema

The table below defines the complete feature schema. Each row corresponds to one entry in the feature vector passed to the classification model. The combination of `message_id` and `source_dataset` acts as the composite unique key for deduplication across training corpora. Optional features are extracted when available but do not block record creation if absent.

Field	Type	Description
<code>message_id</code>	String	RFC 5322 Message-ID header value. Used as primary key.
<code>label</code>	Enum	Ground-truth class: Phish Ham.
<code>predicted_label</code>	Enum	Model output: Phish Ham.
<code>confidence_score</code>	Float [0, 1]	Model probability that the email is Phish.
<code>sender_domain</code>	String	Domain extracted from the From header.
<code>reply_to_domain</code>	String	Domain extracted from Reply-To header. Null if absent.
<code>sender_mismatch</code>	Boolean	True if From domain != Reply-To domain or Return-Path domain.
<code>dkim_pass</code>	Boolean	Result of DKIM signature verification.
<code>spf_pass</code>	Boolean	Result of SPF policy check.
<code>dmarc_pass</code>	Boolean	Result of DMARC alignment check.
<code>urgency_score</code>	Integer	Count of urgency/threat keywords in the body (urgent, verify, expire, suspended, etc.).
<code>url_count</code>	Integer	Total number of hyperlinks found in the email body.
<code>suspicious_url_count</code>	Integer	Number of URLs flagged as suspicious (shortener, IP-address, blacklisted domain, HTTP-only).
<code>url_text_mismatch</code>	Boolean	True if any anchor text visually differs from its href destination.
<code>html_hidden_text</code>	Boolean	True if zero-font or display:none text nodes are detected in the HTML body.
<code>has_attachment</code>	Boolean	True if at least one MIME attachment is present.
<code>attachment_risk</code>	Enum	None Low Medium High. Derived from MIME type and extension.
<code>body_length</code>	Integer	Character count of the plain-text body.
<code>spelling_error_count</code>	Integer (optional)	Number of spelling errors detected. LLM-generated phishing typically scores 0.
<code>brand_impersonated</code>	String (optional)	Brand name detected via keyword + logo analysis (e.g., PayPal, Microsoft, DHL).
<code>top_shap_features</code>	Array of Strings (optional)	Top 5 feature names ranked by absolute SHAP value for this prediction.
<code>source_dataset</code>	String	Name of the dataset from which this email originates (e.g., SpamAssassin, Enron).
<code>scrape_timestamp</code>	DateTime (ISO 8601)	Exact UTC time the email was processed by the detector.

analyst_override	Enum (optional)	Phish Ham Uncertain. Manual label assigned by a SOC analyst after review.
notes	String (optional)	Free-text field for analyst observations or attack campaign identifiers.

References

- [1] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830, 2011.
- [2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*, 2019.
- [3] Richardson, L. Beautiful Soup Documentation. [Online]. Available: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- [4] Hugging Face. Transformers Documentation. [Online]. Available: <https://huggingface.co/docs/transformers>
- [5] Cormack, G. V. Email Spam Filtering: A Systematic Review. *Foundations and Trends in Information Retrieval*, 1(4), 335–455, 2007.
- [6] OpenPhish. Phishing Intelligence Feed. [Online]. Available: <https://openphish.com>
- [7] DMARC.org. Domain-based Message Authentication, Reporting & Conformance. [Online]. Available: <https://dmarc.org>
- [8] Lundberg, S. M., & Lee, S.-I. A Unified Approach to Interpreting Model Predictions. *NeurIPS*, 2017.